Efficiency Optimization Through Symbolic–Subsymbolic AI Integration: An Empirical Energy and Performance Study

Jason Rietzke[0009−0005−2150−6507] and Prof. Dr. Stefan Naumann[0009−0000−6542−2229]

Abstract The productive deployment of large language models (LLMs) for the purpose of workflow execution incurs substantial energy overhead that is largely avoidable for processes with well-defined decision logic. This paper empirically investigates whether restricting an LLM to pure language extraction – combined with a symbolic rule system for process execution – measurably reduces execution time and GPU energy consumption compared to a fully LLM-based approach. Using the ANA emergency call protocol from the SPELL research project as a realistic process workflow, we conducted 30 controlled measurement runs per strategy on identical hardware. Our results demonstrate that the hybrid approach reduces GPU inference energy by a factor of 8.9 and execution time by a factor of 7.7. We decompose this gain into two contributing factors – turn count reduction ($\times$1.87) and per-turn energy reduction ($\times$4.75) – and show that the symbolic processing overhead is energetically negligible (factor $\approx$2,200). These findings provide empirical grounding for the principle that LLMs should be confined to language interpretation tasks while formally specifiable decision logic is delegated to symbolic components – benefiting both energy efficiency and execution time while improving process execution reliability.

Keywords Symbolic AI · Subsymbolic AI · Large Language Models · Energy Efficiency · Green AI · Hybrid AI · Process Execution

---

Jason Rietzke
LiveReader GmbH, Zur Imweiler Wies 3, 66649 Oberthal, Germany; Trier University of Applied Sciences, Environmental Campus Birkenfeld, 55768 Hoppstädten-Weiersbach, Germany
e-mail: `jason.rietzke@livereader.com`

Prof. Dr. Stefan Naumann
Trier University of Applied Sciences, Environmental Campus Birkenfeld, 55768 Hoppstädten-Weiersbach, Germany
e-mail: `s.naumann@umwelt-campus.de`

1 Introduction

Large language models (LLMs) have achieved remarkable performance across a wide range of natural language understanding tasks and are increasingly deployed as general-purpose reasoning engines in production systems. This trend, however, carries a substantial resource cost: as early as 2019, Strubell et al. [9] showed that training large NLP models can consume energy comparable to long-haul air travel. More recent work has shifted attention from training to inference – the operational phase of deployed models – which, in aggregate deployments, constitutes the dominant share of the total energy footprint [3].

Many practical process automation use cases can be decomposed into two structurally distinct subtasks: the interpretation of free-form, natural-language inputs, and the execution of well-defined process steps within a workflow. When such processes are driven entirely by an LLM, the model inevitably takes on decision tasks for which deterministic, rule-based systems would be far more resource-efficient [7]. Whether a deliberate task separation between both paradigms yields measurable efficiency gains is the question this paper addresses.

1.1 Motivation

The energy cost of LLM inference scales superlinearly with model size. Husom et al. [4] demonstrate that models with 70 billion parameters consume approximately 100 times more energy per inference request than 7-billion-parameter models on comparable tasks. At the same time, LLMs are increasingly applied to tasks that are formally fully specifiable and could be handled by rule-based systems at a fraction of the energy cost [7]. Delegating formally deterministic subtasks to an LLM is therefore not merely a technical inefficiency – it constitutes a societal resource question: every unnecessarily delegated decision generates avoidable energy and climate costs [9].

1.2 Research Question and Evaluation Concept

This paper addresses the following research question:

> *Can the targeted use of an LLM exclusively for extracting structured information from natural-language inputs – combined with a symbolic rule system for process execution – achieve a measurable reduction in execution time and energy consumption compared to a fully LLM-based approach?*

To answer this, both approaches are implemented against identical scenarios and empirically measured with respect to execution time and GPU energy consumption. The evaluation is conducted with the Qwen3:30B model on identical hardware under controlled conditions. The measured values are technology-specific and are not directly transferable to other language models – in particular, larger models exhibit substantially higher energy consumption. However, the underlying principle of task separation between LLM extraction and symbolic process control is architecturally general. This paper is designed as a proof of concept: it establishes the existence and order of magnitude of the efficiency gain under controlled conditions, providing empirical grounding for the architectural principle rather than claiming uniform generalizability across all protocols, models, or deployment contexts.

1.3 Paper Structure

Sections 2 and 3 cover foundations and related work. Section 4 describes system design and measurement setup; Section 5 presents results; Section 6 discusses findings, limitations, regulatory implications, and alignment with the UN Sustainable Development Goals; Section 7 concludes with an outlook on future work.

2 Foundations

2.1 Symbolic AI

Symbolic AI systems represent knowledge in explicit, machine-interpretable form – as rules, facts, predicates, or ontologies [7]. Inferences are produced through deterministic rule application, using forward chaining or backward chaining. These properties make symbolic systems fully transparent and auditable: every decision can be traced back to a concrete rule chain.

From a resource perspective, rule-based systems are highly efficient: evaluating a rule set requires a fraction of the computational operations needed for neural inference. Their applicability, however, is constrained to the explicitly modeled knowledge base – they cannot generalize beyond their rules.

2.2 Subsymbolic AI and Large Language Models

Subsymbolic AI systems, in particular LLMs, learn implicit representations of language patterns and semantic relationships from large training corpora, adjusting billions of numerical parameters during training [7]. This grants them remarkable flexibility: they can summarize, classify, extract information from, and reason over text – without explicitly coded rules.

This flexibility comes at a significant cost: the energy consumption of LLM inference correlates strongly with the number of generated output tokens and scales superlinearly with model size [4, 3] (cf. Sect. 3.1).

2.3 Hybrid Approaches

Hybrid AI systems combine both paradigms to exploit their complementary strengths [6, 7]. Typical integration patterns include neural preprocessing for symbolic systems, symbolically informed training, or tight coupling of both layers through differentiable architectures.

For this work, a specific integration pattern is central:

Definition (LLM-as-Extractor Pattern). An LLM is restricted exclusively to extracting structured information from natural-language input and returning it in a defined format to a downstream component; all decision logic is executed by a symbolic rule system. The LLM acts as a language interface, not as a decision engine.

This pattern maximizes efficiency by employing the LLM only for the subtask that has no rule-based alternative, while delegating computationally intensive decision steps to the significantly more efficient symbolic system. Velasquez et al. [10] quantify the theoretical efficiency potential of such architectures (cf. Sect. 3.2).

2.4 The Notitia Symbolic Reasoning Engine

Notitia is a Node.js-based symbolic reasoning environment developed by LiveReader GmbH that interprets formal process graphs as active state machines. A process is encoded as a semantic network of typed nodes – questions, attributes, tasks, and transition conditions – connected by directed edges. At runtime, Notitia evaluates the current process state against all applicable transition conditions and emits the next open task via a Server-Sent Events (SSE) stream; attribute values are written back by external components via an HTTP PATCH interface, triggering re-evaluation. This event-driven execution model keeps the symbolic engine fully decoupled from the component responsible for filling attribute values – a property that makes it directly suitable as the process-control layer in the LLM-as-Extractor pattern.

Notitia is a proprietary product of LiveReader GmbH, the first author's employer. Its use in this study is as a concrete instantiation of the symbolic process-control role; the architectural findings are not specific to Notitia and apply to any comparable rule-based process engine. A detailed description of its integration with the ANA protocol is given in Sect. 4.4.

2.5 Efficiency Metrics

Two metrics are collected for the comparative evaluation of both strategies.

Execution time. Wall-clock time of a complete protocol run, in seconds.

Energy consumption. Measured in Wh or mWh, separately for GPU LLM inference and CPU-side processing. GPU energy is captured via the NVML hardware counter (`nvmlDeviceGetTotalEnergyConsumption`) with 1 mJ resolution directly at the hardware level; CPU consumption of Notitia is estimated via a TDP-based model. The measurement environment and hardware configuration are described in Sect. 4.5.

3 Related Work

3.1 Energy Consumption of Large Language Models

Strubell et al. [9] established the systematic study of AI energy consumption by quantifying, for the first time, the training cost of large NLP models. While early work focused predominantly on training costs, the inference phase has increasingly attracted attention, as it constitutes the dominant energy component in production deployments across large user bases.

Fernandez et al. [3] investigate inference energy consumption systematically across model sizes, quantization levels, and batching strategies. They show that targeted inference optimizations – including quantization, continuous batching, and speculative decoding – can reduce consumption by up to 73 % relative to unoptimized baselines, while the absolute consumption remains substantial even after optimization. The study additionally examines the relationship between hardware utilization and energy efficiency, finding that underutilized GPU capacity – common in single-process deployments – significantly inflates per-request energy cost.

Husom et al. [4] provide detailed inference energy profiling through the MELODI framework, identifying the number of generated output tokens as the primary driver of consumption (Pearson correlation $r = 0.846$). They further establish that 70-billion-parameter models consume approximately 100 times more energy per request than 7-billion-parameter models on comparable tasks – a scaling relationship directly relevant to the model-substitution discussion in Sect. 6.2.

Energy measurement methods. Hardware-level energy measurement for LLM workloads can be performed through several interfaces, each with distinct coverage and granularity. NVML (NVIDIA Management Library) reports cumulative energy consumption of the entire GPU board – including memory, cooling electronics, and board components – with millisecond-level temporal resolution and 1 mJ precision. Intel RAPL (Running Average Power Limit) provides CPU and DRAM power readings at sub-millisecond granularity on supported Intel platforms, but is unavailable on AMD CPUs and offers no GPU coverage. IPMI (Intelligent Platform Management Interface) measures total system power at the baseboard level, capturing the full server draw, but at lower temporal resolution (typically 1–10 Hz) and without per-component attribution. The present study uses NVML for GPU measurements, providing the highest available accuracy for the inference workload of interest. For CPU-side measurement of Notitia –

where RAPL is unavailable on the deployed hardware – a TDP-based estimate is used (cf. Sect. 4.5).

Research gap. The existing body of work on LLM inference energy is almost exclusively focused on single-architecture baselines: varying model size, quantization, or batching strategy within a fully neural pipeline. Empirical comparisons that directly measure the energy consumption of a hybrid symbolic-neural approach against a fully LLM-driven baseline on a realistic process workload are, to the best of the authors' knowledge, absent from the literature. This paper addresses that gap.

3.2 Efficiency Potential of Hybrid AI Approaches

The theoretical basis for efficiency gains through hybrid architectures has been established by several lines of work. Velasquez et al. [10] position neurosymbolic systems as a conceptual counterpoint to resource-intensive scaling strategies, arguing that targeted integration of symbolic components can yield efficiency gains of up to two orders of magnitude without sacrificing the expressive power of subsymbolic processing.

Platzer [6] describes, under the concept of *Intersymbolic AI*, a formal framework for interlinking symbolic and subsymbolic systems, emphasizing that the complementarity of both paradigms should be deliberately exploited: symbolic precision where decisions can be formally specified, subsymbolic flexibility where unstructured data must be processed.

De la Rosa and Tobar [7] categorize hybrid approaches for sequential decision problems and identify efficiency – alongside interpretability and robustness – as a central evaluation criterion. Notably, the preponderant share of the literature grounds efficiency claims for hybrid systems in theoretical arguments or demonstrates them in narrow benchmark scenarios. Empirical measurements that quantify energy savings under realistic, production-like conditions remain rare.

3.3 AI Support in Emergency Dispatch

The SPELL research project (2021–2024), funded by the German Federal Ministry for Economic Affairs and Climate Action, developed a semantic platform for intelligent decision support in emergency dispatch centers [8]. Within the project, dispatch protocols and operational processes were formally encoded in a semantic network – including the ANA protocol (*Adaptive Notrufassistenz*, Adaptive Emergency Call Assistance), a structured dialogue procedure for telephonic initial assessment of emergencies following the German 112 call structure. ANA combines natural-language input processing with fully formally specified protocol logic, making it a well-suited testbed for the LLM-as-Extractor pattern. The technical implementation is described in Sect. 4.

4 System Design and Methodology

4.1 Investigation Subject: The ANA Protocol

The ANA protocol guides a dispatcher through a standardized patient assessment following the 112 call structure [8]. The protocol logic is encoded as a semantic network with approximately 7,800 nodes, describing processes, tasks, transition conditions, and typed attributes.

A complete ANA run traverses a sequence of assessment modules activated by previously captured patient attributes: EREIGNIS → MELDEND → ABCDE → module-specific routing (FAST / HERZINFARKT / STURZ) → OPQRST → SAMPLER → VITALDATEN → PATIENT_ZUGANG → ABSCHLUSS. Each module consists of structured questions with typed output attributes (free text, numeric, boolean, or enum values from a fixed vocabulary).

It should be explicitly noted that an autonomous LLM-based emergency call system would be classified as a high-risk AI system under the EU AI Act [1] and could not be operated in practice in the form examined here – the requirements for transparency, auditability, and human oversight are not met by the fully LLM-driven approach. The baseline strategy serves exclusively as an empirical reference point for quantifying energy demand under realistic conditions.

4.2 Evaluation Scenario

To maintain direct comparability across measurements, a fixed, pre-scripted call scenario is used: Erika Bergmann, 67 years old, has fallen on the stairs and is lying in the hallway. She reports severe pressure in the chest radiating to the left arm; she is pale and diaphoretically sweating; speech is slightly dysarthric. Medical history includes hypertension, type 2 diabetes, anticoagulation therapy (Marcumar), and a myocardial infarction with coronary stent implantation two years prior.

The scenario was selected for its maximal module coverage: the stroke screening (FAST), the cardiac chest pain pathway (HERZINFARKT schema via OPQRST), and the fall module are each fully traversed. All caller responses are statically encoded in a script file and are identical for both strategies – the only variable component is the system architecture under test.

4.3 Strategy A: Fully LLM-Based Approach

In Strategy A, the LLM controls the complete ANA dialogue state. Each conversational turn receives a system prompt (~4,000 tokens) containing the protocol as natural-language rules, a JSON state schema, the conversation history, and the current caller response. The model updates the call state, determines the next dialogue step, and generates the question text; the output is a JSON object containing state, next step, and formulated question. Reasoning steps are emitted in a <think> block (Qwen3 thinking mode); this block is removed from the conversation history but counts fully toward energy consumption.

Table 1 LLM configuration – Strategy A

Parameter	Value
Model	Qwen3:30B (Q4 quantization, Dense-MoE)
Thinking mode	enabled
Temperature	0.1
Input tokens / turn	~3,800–4,200

The LLM signals turn completion autonomously; a safety limit of 60 turns prevents infinite loops. Turn count may vary, as the model decides on module transitions autonomously.

4.4 Strategy B: Hybrid Approach with Notitia

In Strategy B, Notitia (cf. Sect. 2.4) controls navigation of the ANA protocol graph. The LLM is reduced to a pure extraction role. For each protocol question, it receives a minimal, stateless prompt containing only the question text, the caller response, and the attribute types to be extracted. It returns a compact JSON array ({`"values": ["value1", "value2"]`}). The extracted values are written to the process graph via an HTTP PATCH request; Notitia re-evaluates transition conditions and emits the next question.

```
// Notitia -- extraction loop (pseudocode)
for each open_task received via SSE stream:
    prompt = build_prompt(
        question        = open_task.question,
        caller_response = script.next_answer(),
        attribute_types = open_task.attributes
    )
    // ~150 - 300 input tokens (no history, no state schema)
    response = llm.complete(prompt)
    values   = parse_json(response.values)
    graph.patch(open_task.id, values)   // HTTP PATCH to process graph
    // Notitia re-evaluates transitions -> emits next open_task via SSE
```

A typical extraction prompt in Strategy B spans approximately 150–300 input tokens, compared to approximately 3,800–4,200 input tokens per turn in Strategy A. The reduction eliminates the full protocol rule set, the JSON state schema, and the growing conversation history from the LLM context – the primary driver of per-turn energy savings (cf. Sect. 5.3).

Table 2 LLM configuration – Strategy B

Parameter	Value
Model	Qwen3:30B (Q4 quantization, Dense-MoE)
Thinking mode	disabled
Temperature	0.1
Input tokens / call	~150–300

4.5 Measurement Environment and Procedure

Hardware. All measurements are conducted on a dedicated compute server equipped with an NVIDIA RTX Pro 4500 (32 GB GDDR7, PCIe 5.0) as the inference GPU. The model is served via Ollama and remains fully loaded in VRAM between measurement runs, so no cold-start energy contributes to the measurements.

GPU energy measurement. GPU energy is measured via the NVML hardware counter (`nvmlDeviceGetTotalEnergyConsumption`), which reports the cumulative consumption of the entire GPU board (memory, cooling, board electronics) at 1 mJ hardware-level resolution. The counter is read at the start and end of each turn (Strategy A) or each extraction call (Strategy B); the difference yields per-turn consumption. For Strategy B, GPU consumption during Notitia intervals – the GPU draw between LLM calls when the model resides in VRAM but no tensor computation occurs – is additionally captured as the difference between total GPU consumption and the sum of LLM-call measurements.

CPU energy measurement. CPU energy consumption of Notitia (Strategy B) is estimated via a TDP-based model, as no direct hardware energy counter for the CPU is available on the deployed hardware.

Procedure. Each strategy is evaluated in $n = 30$ randomized measurement runs, each preceded by a warm-up run and with a 10-second pause between runs. The randomized order eliminates systematic effects from thermal drift and OS-level caching; thermal throttling was excluded over the complete measurement series. Statistical analysis uses mean, standard deviation, and 95 % confidence interval ($1.96 \times \sigma/\sqrt{n}$).

5 Evaluation

5.1 Execution Time and Turn Count

Strategy B completes a full ANA run in a mean of 71.5 s; Strategy A requires 547.6 s – a factor of **7.7** (Table 3). The difference results from two compounding effects: Notitia traverses the protocol graph in exactly 22 deterministic turns, while Strategy A decides on module transitions autonomously and requires 31–54 turns ($\sigma = 6.2$). Beyond turn

Table 3 Execution time and turn count; mean ± 95 % CI; [min. . . max] = observed range; n = 30

Metric	Strategy A	Strategy B
Turn count (mean)	41.1 ± 2.2 [31. . . 54]	**22.0** ± **0.0** [22. . . 22]
Execution time (s)	547.6 ± 51.9 [383.7. . . 1,133.4]	**71.5** ± **2.2** [56.6. . . 85.5]
Factor	**7.7× faster (Strategy B)**	

count, per-turn latency also differs substantially – Strategy A generates full JSON state plus a thinking block per turn, while Strategy B produces only a compact value array. The higher standard deviation in Strategy A (±51.9 s vs. ±2.2 s) reflects inherent non-determinism; the non-overlapping 95 % CIs (A: ≥495.7 s; B: ≤73.7 s) confirm the statistical reliability of the group difference.

5.2 Energy Consumption

Table 4 GPU and CPU energy consumption; mean ± 95 % CI, n = 30

Metric	Strategy A	Strategy B
GPU energy – LLM inference (Wh)	28.92 ± 1.85	**3.25** ± **0.12**
GPU energy – total (Wh)	–	3.96 ± 0.11
GPU energy – Notitia intervals (mWh)	–	706.4 ± 9.6
GPU energy per turn (mWh)	701.7 ± 14.0	**147.7** ± **5.4**
CPU energy – Notitia (mWh)	–	1.47 ± 0.26
Factor GPU-LLM	**8.9× lower (Strategy B)**	

GPU inference energy in Strategy A is 28.92 Wh; in Strategy B it is 3.25 Wh – a factor of **8.9** (Table 4). Relative to Strategy B's total GPU consumption (LLM inference plus GPU draw during Notitia intervals, 3.96 Wh), the factor is **7.3**. In Strategy A, no Notitia intervals exist; the measured LLM consumption (28.92 Wh) equals the total GPU draw for the run.

GPU consumption during Notitia intervals (706.4 mWh, ~18 % of total GPU consumption in Strategy B) arises because the model remains fully loaded in VRAM and draws approximately 35.8 W even during idle periods – a fixed overhead that scales proportionally with total runtime.

CPU consumption of Notitia amounts to 1.47 mWh per run (0.067 mWh/turn), compared to 147.7 mWh/turn for LLM extraction – a factor of ~**2,200**. The symbolic component is thus energetically negligible.

5.3 Decomposition of the Efficiency Gap

The factor of 8.9 in GPU energy consumption can be attributed to two directly measurable partial factors:

Turn count. Strategy A requires a mean of 41.1 turns; Strategy B uses exactly 22 – a factor of ~1.87. Since each turn requires a complete LLM call, this difference multiplies directly onto total energy consumption.

GPU energy per turn. Per-turn GPU consumption is 701.7 vs. 147.7 mWh – a factor of ~4.75. Strategy A processes the full JSON state (~1,500 tokens), question text, and conversation history per turn and additionally executes a <think> block that is fully computed on the GPU without producing protocol-relevant output. Strategy B returns only a compact value array (50–200 tokens); thinking mode is disabled since pure field extraction requires no protocol reasoning. Both token volume and thinking overhead contribute to this factor; NVML does not attribute them separately, so their individual shares cannot be isolated. Thinking mode is enabled in Strategy A and disabled in Strategy B as a deliberate design choice reflecting the task requirements of each role: autonomous protocol navigation benefits from extended reasoning, while stateless field extraction does not. The thinking overhead is therefore an inseparable property of the fully LLM-driven architecture as deployed in practice, not an independently variable parameter.

Fig. 1 Multiplicative decomposition of the 8.9× factor; Strategy A vs. B

Both factors multiply: $\sim 1.87 \times \sim 4.75 \approx 8.9$ (Fig. 1). The decomposition confirms that only their combination within the hybrid architecture pattern yields the full gain – no isolated measure would achieve the same effect. The architectural separation additionally opens a second, independent optimization dimension – model size substitution – which is discussed in Sect. 6.2.

6 Discussion

6.1 Interpretation of Results

The measured efficiency gains confirm the core hypothesis: restricting the LLM to its genuine strength – the interpretation of natural language – while delegating protocol logic to a symbolic system reduces GPU inference energy by a factor of 8.9 and execution time by a factor of 7.7. This finding is consistent with the theoretical positions of Velasquez et al. [10] and Platzer [6], who position neurosymbolic systems as a resource-efficient alternative to purely scale-based approaches – and, to the best of the authors' knowledge, provides the first controlled empirical quantification of this principle for a realistic process workload.

Notably, the efficiency gains are not attributable to a single cause but emerge from three mutually reinforcing factors (cf. Sect. 5.3). This implies that partial optimizations – e.g., prompt shortening without task separation – would realize only a fraction of the achievable gain.

The CPU overhead introduced by Notitia (cf. Sect. 5.2) is energetically negligible.

6.2 Model Substitution Potential

Both strategies in this study deliberately employ the same model (Qwen3:30B) to ensure a controlled, isolated comparison of architectural decisions. The measured GPU efficiency of Strategy B is therefore a conservative lower bound: the extraction subtask – reading a caller response and filling typed attributes – is semantically far simpler than the full protocol reasoning performed in Strategy A, and could be handled by a significantly smaller model without loss of quality.

Drawing on the scaling relationship established in Sect. 3.1 [4], a 3–7B extraction model could reduce the LLM inference energy of Strategy B by a further order of magnitude. The interpolation treats the empirical 70B/7B energy ratio as a conservative proxy for the 30B-to-7B step: since Qwen3:30B lies closer to the 7B boundary in parameter count, the actual gain is likely to exceed this estimate. Log-linear scaling between model size and per-request energy is assumed, consistent with [4], but remains model-family-specific; the projected values in Table 5 should be read as order-of-magnitude estimates.

The projections in Table 5 are intentionally conservative and do not account for fine-tuned extraction models, which may show even lower energy consumption for this specific subtask. Crucially, this second optimization dimension – model size selection – is architecturally decoupled from the gains measured in this paper: it becomes available precisely because the symbolic layer absorbs all process control, freeing the LLM role to be filled by any model capable of structured field extraction. The two dimensions are therefore additive, not competing.

Table 5 Measured and projected GPU inference energy; projections extrapolate the 70B/7B scaling factor from [4]

Configuration	GPU LLM (Wh)	Factor vs. A	Basis
Strategy A – 30B, full LLM	28.92	1.0×	measured
Strategy B – 30B, hybrid	3.25	8.9×	measured
Strategy B – 7B, hybrid	~0.35	~83×	projected
Strategy B – 3B, hybrid	~0.15	~193×	projected

6.3 Limitations

The measurements are based on a single, pre-scripted scenario. Scenarios with lower module coverage would reduce the absolute difference; scenarios with greater complexity or more protocol layers would increase it. Generalizability of the results to other protocols, models, or hardware configurations is not immediate and requires further investigation.

As discussed in Sect. 1.2, the results are technology-specific; the correctness of either approach is outside the scope of this study. The LLM-based approach is subject to known risks – hallucination, prompt sensitivity, inconsistent state management across many turns – which were not systematically evaluated here. Extraction quality in Strategy B was also not formally measured: an extraction error would not increase energy consumption but would corrupt the downstream decision of the hybrid approach.

Reproducibility. Exact replication of Strategy B requires access to Notitia and the ANA protocol graph from the SPELL project. As Notitia is a proprietary system and the SPELL protocol graph has not been publicly released, neither component is available for independent replication. However, the architectural pattern itself – a symbolic graph engine receiving structured output from a stateless LLM extraction prompt – is implementable with any comparable symbolic reasoning engine such as Apache Jena or Protégé with a DL reasoner (e.g., HermiT) provide a viable replication path. The LLM model (Qwen3:30B via Ollama), measurement methodology (NVML hardware counter, $n = 30$ randomized runs), and evaluation scenario (fully specified in Sect. 4.2) are publicly reproducible. Conceptual replication with a different protocol and a comparable symbolic engine is therefore feasible; exact numerical replication requires re-implementation of Strategy B's components.

GPU consumption during Notitia intervals (~18 % of total GPU consumption in Strategy B) demonstrates that loading a large model into VRAM incurs a fixed overhead that is not amortized at low call frequencies. In parallel-process deployments or continuous dispatch center operation, this share would decrease proportionally.

6.4 Threats to Validity

Internal validity. All measurements used identical hardware, the same model weights, and a fixed random seed per run, minimizing the risk of systematic bias. The randomized run order and warm-up procedure reduce thermal and cache effects. The primary internal threat is the TDP-based CPU energy estimate for Strategy B: as an upper bound derived from thermal design power, it may overstate actual CPU consumption, but given its negligible magnitude (1.47 mWh vs. 3,250 mWh GPU inference), this uncertainty does not affect any reported efficiency factor.

External validity. The evaluation uses a single, pre-scripted call scenario on a single model (Qwen3:30B). Efficiency factors reported here are specific to this configuration; other models, prompt formulations, or protocol structures may yield different values. The decomposition of the factor 8.9 into turn-count ($\times 1.87$) and per-turn energy ($\times 4.75$) components provides a framework for estimating results under different configurations without requiring additional measurements, but such extrapolations carry inherent uncertainty.

Construct validity. The study measures GPU inference energy via NVML, which captures the full GPU board draw including memory and idle power – not raw compute energy alone. This is the correct construct for operational energy footprint, but comparisons to studies using compute-only metrics require adjustment.

6.5 Implications for Sustainable AI Architectures

The results provide an empirical argument for deliberate task separation: LLMs should be confined to interpreting unstructured language – not extended to deterministic decision logic. In the examined scenario, this separation not only substantially reduces energy consumption but also increases predictability and auditability, as Notitia's protocol logic remains fully transparent.

For safety-critical application domains such as emergency management, this is not solely a question of efficiency. The EU AI Act [1] imposes explicit regulatory requirements on high-risk AI systems that are directly relevant here. Article 9 mandates a risk management system proportionate to the risk profile of the application; Notitia's deterministic, fully traceable decision logic is inherently more amenable to risk management than a stochastic LLM making autonomous protocol decisions. Article 13 requires that high-risk AI systems be designed with transparency sufficient for users to interpret outputs; the symbolic protocol graph provides full interpretability of each decision step and transition condition, while an LLM's internal reasoning is opaque. Article 14 mandates effective human oversight – in the hybrid approach, Notitia's deterministic state machine can be paused, inspected, or overridden at any point, whereas an LLM-driven approach requires additional architectural effort to expose equivalent oversight capabilities. The LLM-as-Extractor pattern thus simultaneously addresses efficiency and compliance requirements, making it a pragmatic default for process-driven applications in regulated sectors.

6.6 Alignment with Sustainable Development Goals

EnviroInfo 2026 explicitly maps submitted contributions to the United Nations Sustainable Development Goals (SDGs). The findings of this study provide direct and quantifiable connections to two goals.

SDG 13 – Climate Action. The empirically documented 8.9× reduction in GPU inference energy per protocol run corresponds to a proportional reduction in operational CO_2-equivalent emissions. The International Telecommunication Union (ITU) estimates that AI workloads already account for 1–2 % of global electricity consumption as of 2025, a share growing faster than aggregate digitalization [5].

To illustrate the climate-relevant scale: using the European average grid emission factor of approximately 250 g CO_2eq/kWh [2], a single ANA protocol run in Strategy A (28.92 Wh = 0.029 kWh) generates approximately 7.2 g CO_2eq. Strategy B (3.25 Wh LLM inference) generates approximately 0.8 g CO_2eq – a reduction of 6.4 g per run. At 1,000 protocol runs per day (a realistic load for a medium-sized dispatch center), this corresponds to savings of 6.4 kg CO_2eq per day, or approximately 2.3 tonnes per year, solely from the architectural change – before any hardware upgrade or renewable energy sourcing. Energy-efficient AI architectures of the type demonstrated here – achievable without hardware replacement, model retraining, or reduction in functional capability – represent a practical and immediately deployable lever for reducing the climate footprint of digital services.

SDG 7 – Affordable and Clean Energy. Resource-efficient AI reduces the energy demand of digital infrastructure at scale. Emergency dispatch centers and other critical infrastructure are often operated in resource-constrained environments or in regions with limited access to renewable energy sources. The ability to deploy AI-assisted process execution at 11 % of the GPU energy cost of a fully neural approach – without sacrificing functional quality – makes such systems accessible to a broader range of operators and deployment contexts. This democratization of energy-efficient AI aligns with SDG 7's emphasis on universal access to affordable, reliable, and modern energy services.

7 Conclusion and Future Work

This paper has empirically investigated whether the deliberate restriction of an LLM to structured data extraction – combined with a symbolic system for protocol control – yields measurable efficiency gains. Using the ANA emergency call protocol from the SPELL project and $n = 30$ controlled measurement runs on identical hardware, a reduction factor of **8.9** in GPU inference energy and **7.7** in execution time was measured.

The measurements additionally quantify the superproportional efficiency of symbolic processing: 0.067 mWh CPU consumption of Notitia per turn stands against 147.7 mWh GPU consumption of LLM extraction (factor ~**2,200**); compared to Strategy A (701.7 mWh/turn), this difference reaches approximately **four orders of magnitude**. This underscores a central design principle: every formally specifiable process component should be delegated to a symbolic component – not because LLMs could not handle it, but because the energy cost is orders of magnitude higher.

The results validate the *LLM-as-Extractor* architectural pattern, not the specific implementation used here. Notitia serves as a concrete instantiation; the measured efficiency gains are a property of the pattern – the structural separation of language extraction from deterministic process control – and are expected to hold for any functionally equivalent symbolic engine.

The architectural task separation further opens a second, independent optimization dimension: the LLM in Strategy B can be replaced by smaller, task-specialized models without modifying Notitia. Based on scaling data from Husom et al. [4], a 3–7B extraction model could reduce the LLM energy contribution of Strategy B by a further order of magnitude, potentially reaching total GPU inference energy below 0.2 Wh per ANA run.

Future work should address: the formal evaluation of extraction quality for smaller models and its impact on protocol correctness; the transferability of the LLM-as-Extractor architecture to further domain-specific protocols – such as medical anamnesis systems or rule-based administrative procedures; and the investigation of energy consumption and throughput under parallel operation of multiple processes on shared infrastructure.

Acknowledgements The ANA protocol and semantic network used in this study originate from the SPELL research project (2021–2024), funded by the German Federal Ministry for Economic Affairs and Climate Action (BMWK/BMWi). The authors thank the SPELL project consortium for making the protocol material available.

Parts of this manuscript were drafted with the assistance of large language model tools (Claude by Anthropic). The research design, all measurements, data analysis, interpretation, and the intellectual content of this work are entirely the authors' own.

References

1. European Parliament and Council of the European Union: Regulation (EU) 2024/1689 of the european parliament and of the council of 13 june 2024 laying down harmonised rules on artificial intelligence (artificial intelligence act). Official Journal of the European Union, L series (2024). URL `https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689`
2. Eurostat: Greenhouse gas emission intensity of electricity generation in Europe. Eurostat Statistics Explained (2023). URL `https://ec.europa.eu/eurostat/statistics-explained/index.php/Greenhouse_gas_emission_intensity_of_electricity_generation_in_Europe`
3. Fernandez, J., Na, C., Tiwari, V., Bisk, Y., Luccioni, S., Strubell, E.: Energy considerations of large language model inference and efficiency optimizations (2025). URL `https://arxiv.org/abs/2504.17674`. ArXiv preprint arXiv:2504.17674

4. Husom, E.J., Goknil, A., Shar, L.K., Sen, S.: The price of prompting: Profiling energy use in large language models inference (2024). URL `https://arxiv.org/abs/2407.16893`. ArXiv preprint arXiv:2407.16893, SINTEF Digital
5. International Telecommunication Union: Environmental efficiency for AI and other emerging technologies. ITU-T Focus Group on Environmental Efficiency for AI and Other Emerging Technologies (FG-AI4EE), Final Report (2025). URL `https://www.itu.int/pub/T-TUT-ENER-2025`
6. Platzer, A.: Intersymbolic AI: Interlinking symbolic AI and subsymbolic AI (2024). URL `https://arxiv.org/abs/2406.11563`. ArXiv preprint arXiv:2406.11563
7. de la Rosa, J., Tobar, F.: A review of symbolic, subsymbolic and hybrid methods for sequential decision making. ACM Computing Surveys (2024). DOI 10.1145/3663366
8. SPELL Consortium: SPELL — semantic platform for intelligent decision and deployment support in emergency dispatch centers. Research project, funded by the German Federal Ministry for Economic Affairs and Climate Action (BMWK) (2024). URL `https://spell-plattform.de`
9. Strubell, E., Ganesh, A., McCallum, A.: Energy and policy considerations for deep learning in NLP. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 3645–3650 (2019). DOI 10.18653/v1/P19-1355
10. Velasquez, A., Bhatt, N., Topcu, U., Wang, Z., Sycara, K., Stepputtis, S., Neema, S., Vallabha, G.: Neurosymbolic AI as an antithesis to scaling laws. PNAS Nexus **4**(5) (2025). DOI 10.1093/pnasnexus/pgaf117